Design, Construction, and Application of a Generic Visual Language Generation Environment

Kang Zhang, *Senior Member*, *IEEE*, Da-Qian Zhang, and Jiannong Cao, *Senior Member*, *IEEE*

Abstract—The implementation of visual programming languages (VPLs) and their supporting environments is time consuming and tedious. To ease the task, researchers have developed some high level tools to reduce the development effort. None of these tools, however, can be easily used to create a complete visual language in a seamless way like the lex/yacc tools for textual language constructions. This paper presents the design, construction, and application of a generic visual language generation environment, called VisPro. The VisPro design model improves the conventional Model-View-Controller framework in that its functional modules are decoupled to allow independent development and integration. The VisPro environment consists of a set of visual programming tools. Using VisPro, the process of VPL construction can be divided into two steps: lexicon definition and grammar specification. The former step defines visual objects and a visual editor, and the latter step provides language grammars with graph rewriting rules. The compiler for the VPL is automatically created according to the grammar specification. A target VPL is generated as a programming environment which contains the compiler and the visual editor. The paper demonstrates how VisPro is used by building a simple visual language and a more complex visual modeling language for distributed programming.

Index Terms—Visual programming, visual languages, language construction, graph grammar.

1 Introduction

Visual programming is aimed at effectively improving the programming productivity by applying visual technologies to support program construction. Visual programming languages (VPLs) have been successfully used in several application areas: teaching children and adults, programming for nonprogrammers, development of user-interfaces, etc. VPLs have also been used in the design and analysis of software systems. Well-known examples of software modeling and specification languages include UML—the Unified Modeling Languages [21], automata, Petri nets, etc.

Implementing a visual language is much harder than implementing a textual language [19]. VPLs are usually embedded and tightly integrated within visual environments. Consequently, they are often characterized by the attributes of the environments [9]. The VPL implementation involves the implementation of a whole programming environment with a user interface which supports developing programs using a visual language. Notice that VPL interfaces are not the same as graphical user interfaces (GUIs) nor are they just for visualization. Traditional GUI development toolkits are inadequate for the creation of VPLs because they do not support syntactic and semantic specifications of visual programming. The graphical user interface of a visual language relates to the language's syntax and semantics. The interaction (dialogue) between the interface, the syntax, and the semantics must be maintained. Implementing a VPL interface and its support for syntactic and semantic specifications of visual programming suffers from a problem common to all large, complex software systems, i.e., the generation is difficult and time consuming. The remaining part of this section will start by a discussion on the related work, then address the current problems that we have identified, followed by our approaches to solving the problems.

1.1 Related Work

Researchers have developed some high level tools to ease the implementation of visual languages and visual programming environments. For example, Escalante supports the construction of applications for visual languages that are based on object-relationship abstractions [17]. It provides mechanisms for iterative design, rapid prototyping, and generation of visual language applications within an integrated environment. DiaGen [18] is a tool for producing diagram editors, which can be used to construct visual programs. These tools can greatly reduce the effort of developing visual languages, although they focus mainly on the construction of user interface aspects of VPLs.

SPARGEN [11] is a visual language compiler generator. Its generated parser supports additional action routines written in C++ and, thus, allows complicated actions to be specified in the form of rules. It, however, does not support the generation of a visual programming environment.

- *K. Zhang is with the Department of Computer Science, University of Texas at Dallas, Box 830668, MS EC31, Richardson, TX 75083-0688. E-mail: kzhang@utdallas.edu.*
- *D.-Q. Zhang is with the Corel Corporation, 621-25 Woodbridge Cres., Nepean, ON K2B 7T4. E-mail: daqianz@corel.ca.*
- *J. Cao is with the Department of Computing, Hong Kong Polytechnic University, Hung Hom, Kowloon, Hong Kong. E-mail: csjcao@comp.polyu.edu.hk.*

Manuscript received 23 Nov. 1998; revised 9 Apr. 1999; accepted 17 June 1999.
Recommended for acceptance by M. Jazayeri.
For information on obtaining reprints of this article, please send e-mail to: tse@computer.org, and reference IEEECS Log Number 108329.

0098-5589/01/$10.00 © 2001 IEEE

[24] G. Rozenburg and E. Welzl, "Boundary NLC Graph Grammars—Basic Definitions, Normal Forms, and Complexity," *Information and Control,* vol. 69, pp. 136-167, 1986.
[25] D. Schefstrom and G. van den Broek, *Tool Integration-Environments and Frameworks.* Chichester, England: John Wiley, 1993.
[26] P. Salis, G. Tate, and S. MacDonell, *Software Engineering.* Addison-Wesley, 1995.
[27] A. Schürr, A. Zundorf, and A. Winter, "Visual Programming with Graph Rewriting Systems," *Proc. 11th IEEE Symp. Visual Languages (VL '95),* pp. 326-333, Sept. 1995.
[28] S. Shatz, *Development of Distributed Software.* New York: Macmillan, 1993.
[29] C. Upson, T. Faulhaver, D. Kamins, D. Laidlaw, D. Schlegel, J. Vroom, R. Gurwitz, and A. Van Dam, "The Application Visualization System: A Computational Environment for Scientific Visualization," *IEEE Computer Graphics and Applications,* vol. 9, no. 7, pp. 30-42, July 1989.
[30] L. Weitzman and K. Wittenburg, "Automatic Presentation of Multimedia Documents Using Relational Grammars," *Proc. ACM Int'l Conf. Multimedia,* pp. 443-451, Oct. 1994.
[31] N. Wilde and C. Lewis, "Spreadsheet-Based Interactive Graphics: From Prototype to Tool," *ACM SIGGHI Special Issue, Proc. CHI '90,* pp. 153-159, Apr. 1990.
[32] D.-Q. Zhang and K. Zhang, "Reserved Graph Grammar: A Specification Tool for Diagrammatic VPLs," *Proc. 13th IEEE Symp. Visual Languages (VL '97),* pp. 284-291, Sept. 1997.
[33] D-Q. Zhang and K. Zhang, "On A Visual Distributed Programming Environment and Its Construction by a Meta Toolset," *Proc. SEKE '98—10th Int'l Conf. Software Eng. and Knowledge Eng.,* June 1998.
[34] D.-Q. Zhang and K. Zhang, "VisPro: A Visual Language Generation Toolset," *Proc. 14th IEEE Symp. Visual Languages (VL '98),* pp. 195-202, Sept. 1998.
[35] K. Zhang, X. Ma, and T. Hintz, "The Role of Graphics in Parallel Program Development," *J. Visual Languages and Computing,* vol. 10, no. 3, pp. 215-243, 1999.



Kang Zhang received his BEng in Computer Engineering from the University of Electronic Science and Technology, China, in February 1982; and PhD from the University of Brighton, UK, in 1990. He is currently an associate professor of computer science at the University of Texas at Dallas. He has held positions as a Lecturer and Senior Lecturer at Macquarie University, Sydney, Australia; a Research Assistant and SERC Postdoctoral Fellow at the University of Brighton, UK; and a Software Engineer at the East-China Research Institute of Computer Technology, Shanghai, China. Dr Zhang's current research interests are in the areas of software visualisation, parallel programming tools, visual programming, and Internet computing. Dr Zhang is a senior member of IEEE.



Da-Qian Zhang received the BEng and MEng degrees in computer engineering from Zhejiang University, China, in 1985 and 1993, respectively; and the PhD degree in computer science from Macquarie University, Sydney, Australia in 1998. He is a software engineer at Corel Corporation, Ottawa, Canada. Dr. Zhang was an assistant lecturer at Hangzhou University, China, for five years, before pursuing his PhD at Macquarie University. After his PhD, he became a software developer at Daedalian System Group, Canada, and then joined Corel Corporation.



Jiannong Cao received the BS degree (1982) from Nanjing University, China, the MS degree (1986), and PhD degree (1990) from Washington State University, all in computer science. He has been on the faculty of computer science at James Cook University (Queensland, Australia), University of Adelaide (South Australia, Australia), and City University of Hong Kong. He joined the Hong Kong Polytechnic University in 1997, where he is currently an assistant professor. Dr. Cao's research interests include parallel and distributed systems, computer networks, Internet computing, fault tolerance, and programming methodology and environments. He has published more than 60 technical papers in the above areas, which have appeared in international journals and conference proceedings. His recent research has focused on how to build high-performance, fault-tolerant distributed systems and applications on the Internet. Dr. Cao is a member of ACM and a senior member of the IEEE.